

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
ФГАОУ ВО «СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»
ИНСТИТУТ ПЕРСПЕКТИВНОЙ ИНЖЕНЕРИИ
ДЕПАРТАМЕНТ ЦИФРОВЫХ, РОБОТЕХНИЧЕСКИХ
СИСТЕМ И ЭЛЕКТРОНИКИ

КУРСОВАЯ РАБОТА
по дисциплине
«ИНТЕРНЕТ ТЕХНОЛОГИИ»
на тему:
«Разработка веб-приложения для ведения заметок.»

Выполнила:
Злобин Владислав Евгеньевич
Студент 4 курса группы ПИН-б-з-22-1
Направления подготовки
09.03.03 Прикладная информатика
заочной формы обучения

(подпись)
Руководитель работы:
Щеголев А.А.
(ФИО)

Работа допущена к защите _____
(подпись руководителя) (дата)

Работа выполнена и
защищена с оценкой _____ Дата защиты _____

Члены комиссии: _____
(должность) (подпись) (И.О. Фамилия)

Ставрополь, 2026 г

УТВЕРЖДАЮ

директор департамента цифровых,
робототехнических систем
и электроники
_____ Азаров И.В.

Институт Перспективной инженерии
Департамент Цифровых, робототехнических систем и
электроники
Направление подготовки 09.03.03
Направленность (профиль) Прикладная информатика

ЗАДАНИЕ

на курсовую работу/проект

студента Злобин Владислав Евгеньевич
(фамилия, имя, отчество)

по дисциплине Интернет-технологии

1. Тема работы «Разработка веб-приложения для ведения заметок»

2. Цель - разработать веб-приложение для ведения заметок, обеспечивающее внесение напоминаний с возможностью редактирования существующего списка заметок.

3. Задачи Сбор и анализ информации: Поиск и изучение литературы, статей, научных исследований и других источников информации по выбранной теме. Планирование работы: Составление плана курсовой работы, определение основных этапов и сроков выполнения. Написание текста: Оформление результатов исследования в виде структурированного текста, включающего введение, основную часть и заключение. Проведение экспериментов и практическая реализация: Если это необходимо, выполнение практических заданий, связанных с разработкой программного обеспечения, созданием веб-сайтов или проведением экспериментов. Оформление работы: Соблюдение требований к оформлению курсовой работы, включая правильное оформление списка литературы, таблиц, рисунков и схем. Представление и защита работы: Подготовка презентации и выступление перед комиссией, где студент демонстрирует результаты своего труда и отвечает на вопросы.

4. Перечень подлежащих разработке вопросов:

а) по теоретической части Исторический обзор развития интернет-технологий (в зависимости от выбранной темы: стека технологий). Основные понятия и определения. Описание выбранного стека технологий. Анализ существующих решений и подходов.

б) по практической части Постановка задачи. Выбор методов и инструментов для решения задачи. Описание процесса разработки. Результаты эксперимента или практической реализации. Оценка эффективности предложенных решений.

5. Исходные данные:

а) по литературным источникам _____

б) по вариантам, разработанным преподавателем _____

в) иное _____

6. Список рекомендуемой литературы _____

7. Контрольные сроки представления отдельных разделов курсовой работы:

25 % - _____ “ ” _____ 20_ г.

50 % - _____ “ ” _____ 20_ г.

75 % - _____ “ ” _____ 20_ г.

100 % - _____ “ ” _____ 20_ г.

8. Срок защиты студентом курсовой работы “ ” _____ 20_ г.

Дата выдачи задания “ ” _____ 20_ г.

Руководитель курсовой работы

Старший преподаватель департамента цифровых, робототехнических систем
и электроники института перспективной инженерии Щеголев А.А.

(ученая степень, звание)(личная подпись)(инициалы, фамилия)

Задание принял к исполнению студент заочной формы обучения 4 курса
ПИН-6-3-22-1 группы _____

(личная подпись)

(инициалы, фамилия)

Оглавление

ВВЕДЕНИЕ	5
1 ТЕОРЕТИЧЕСКИЕ ОСНОВЫ РАЗРАБОТКИ ВЕБ-ПРИЛОЖЕНИЙ	7
1.1 Исторический обзор развития интернет-технологий.....	7
1.2 Основные понятия веб-разработки	8
1.3 Описание выбранного стека технологий	10
1.4 Анализ существующих решений для ведения заметок	12
2 РАЗРАБОТКА ВЕБ-ПРИЛОЖЕНИЯ ДЛЯ ВЕДЕНИЯ ЗАМЕТОК.....	15
2.1 Постановка задачи	15
2.2 Выбор методов и инструментов для решения задачи	15
2.3 Описание объектной модели и структуры данных.....	17
2.4 Процесс разработки серверной части.....	18
2.5 Разработка клиентской части и вёрстка	20
2.6 Тестирование и оценка эффективности	22
2.7 Использование системы контроля версий Git и GitHub	23
ЗАКЛЮЧЕНИЕ	25
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	27

ВВЕДЕНИЕ

Актуальность темы обусловлена повсеместным внедрением интернет-технологий в профессиональную деятельность, образовательный процесс и повседневную жизнь, а также возросшим спросом на системы, позволяющие делать необходимые заметки и запоминать большие объемы информации. Веб-приложения стали неотъемлемой частью цифровой экосистемы, обеспечивая пользователям доступ к информации и сервисам из любой точки мира.

Объект исследования – процесс разработки веб-приложений на основе серверной платформы Node.js и фреймворка Express.

Предмет исследования – методы и инструменты реализации веб-приложения для ведения заметок с хранением данных в памяти сервера, а также организация взаимодействия клиентской и серверной частей.

Цель работы – разработка и документирование веб-приложения для ведения заметок, демонстрирующего практическое применение технологий Node.js, Express, HTML и CSS, а также использование системы контроля версий Git.

Для достижения цели поставлены следующие задачи:

- Провести анализ теоретических основ веб-разработки, включая историю развития интернет-технологий, современный стек технологий и существующие решения для ведения заметок.
- Сформулировать функциональные и нефункциональные требования к разрабатываемому приложению.
- Обосновать выбор инструментальных средств для реализации бэкенда и фронтенда.
- Разработать серверную часть приложения на Node.js с использованием Express, реализующую REST API для управления заметками.
- Разработать клиентскую часть приложения на HTML и CSS с использованием JavaScript для взаимодействия с API.
- Организовать хранение заметок в памяти сервера.

- Использовать систему контроля версий Git для отслеживания этапов разработки и разместить проект на GitHub.

- Провести тестирование функциональности приложения и оценить его эффективность.

Методы исследования: анализ литературных источников и технической документации, сравнительный анализ инструментов, объектно-ориентированное проектирование, программирование на JavaScript (Node.js, клиентский JavaScript), ручное тестирование.

Теоретическая значимость работы заключается в систематизации знаний о современных подходах к разработке веб-приложений на основе Node.js и Express, а также в углублённом анализе архитектуры клиент-серверных приложений.

Практическая значимость состоит в создании работоспособного веб-приложения, которое может быть использовано для личных целей или в качестве учебного примера для студентов, изучающих веб-технологии. Кроме того, опыт, полученный в ходе разработки, применим для более сложных проектов, включающих работу с базами данных и системами аутентификации.

Структура работы соответствует логике исследования и включает введение, две главы (теоретическую и практическую), заключение, список использованных источников и приложения. В первой главе рассматриваются теоретические аспекты веб-разработки, даётся исторический обзор, вводятся основные понятия, описывается выбранный стек технологий и проводится анализ существующих решений для ведения заметок. Во второй главе подробно излагаются этапы практической реализации: постановка задачи, выбор инструментов, проектирование объектной модели, разработка серверной и клиентской частей, тестирование и работа с Git. В заключении подводятся итоги работы, формулируются выводы и намечаются перспективы дальнейшего развития приложения.

1 ТЕОРЕТИЧЕСКИЕ ОСНОВЫ РАЗРАБОТКИ ВЕБ-ПРИЛОЖЕНИЙ

1.1 Исторический обзор развития интернет-технологий

Развитие интернет-технологий неразрывно связано с эволюцией Всемирной паутины (World Wide Web). В 1990 году английский учёный Тим Бернерс-Ли, работая в CERN, создал первый веб-сервер и веб-браузер, что положило начало эпохе статических HTML-страниц. На начальном этапе сайты представляли собой набор гипертекстовых документов, а взаимодействие с пользователем ограничивалось переходами по ссылкам. Протокол HTTP версии 0.9 был крайне прост: он поддерживал только один метод (GET) и не имел заголовков.

С появлением языка JavaScript в 1995 году (разработан Бренданом Эйхом в Netscape) и технологии динамического обновления страниц без перезагрузки (AJAX) в начале 2000-х годов началась эра динамических веб-приложений. JavaScript позволил выполнять код на стороне клиента, обрабатывать события и изменять содержимое страницы без её полной перезагрузки. AJAX (Asynchronous JavaScript and XML) дал возможность отправлять асинхронные HTTP-запросы и обновлять части страницы, что кардинально улучшило пользовательский опыт.

Серверные технологии также эволюционировали. Первые динамические сайты использовали CGI (Common Gateway Interface) – интерфейс, позволявший выполнять внешние программы для обработки запросов. Затем появились языки программирования, специально ориентированные на веб-разработку: PHP (1995), Python (1991, но активно в вебе с конца 1990-х), Ruby (1995). Постепенно сформировались фреймворки, упрощающие разработку: для PHP – Laravel, Symfony; для Python – Django, Flask; для Ruby – Ruby on Rails.

Ключевым событием стало появление платформы Node.js в 2009 году, разработанной Райаном Далем. Node.js позволил использовать JavaScript не только на стороне клиента, но и на сервере. Платформа основана на движке V8 от Google и предлагает событийно-ориентированную неблокирующую

модель ввода-вывода, что делает её эффективной для создания высокопроизводительных приложений реального времени. Вслед за Node.js появился фреймворк Express, ставший де-факто стандартом для построения веб-серверов на этой платформе.

Параллельно развивались архитектурные подходы. От монолитных приложений, где весь код объединён в одном развёртываемом артефакте, перешли к микросервисной архитектуре, позволяющей независимо разрабатывать и масштабировать отдельные сервисы. От серверного рендеринга, где HTML формировался на сервере, перешли к одностраничным приложениям (SPA), где клиент получает JavaScript-код, который сам управляет отображением, а сервер предоставляет только данные через API. В области веб-сервисов SOAP (Simple Object Access Protocol) уступил место REST (Representational State Transfer) и GraphQL.

Сегодня веб-разработка включает широкий спектр инструментов: фронтенд-фреймворки (React, Angular, Vue), бэкенд-фреймворки (Express для Node.js, Django для Python, Spring для Java), системы управления базами данных (SQL и NoSQL), контейнеризацию (Docker), оркестрацию (Kubernetes), системы непрерывной интеграции и доставки (CI/CD). Разработка веб-приложений стала многоуровневой и требует от специалиста знаний в различных областях.

1.2 Основные понятия веб-разработки

Для понимания процесса создания веб-приложений необходимо определить ключевые понятия, используемые в работе.

Веб-приложение – это клиент-серверное приложение, в котором клиентом выступает браузер (или другое приложение, поддерживающее HTTP), а сервером – программное обеспечение, обрабатывающее запросы и формирующее ответы. Веб-приложения работают по протоколу HTTP/HTTPS и могут выполнять сложную бизнес-логику, взаимодействовать с базами данных, обрабатывать файлы, отправлять электронные письма и т.д.

Клиентская часть (frontend) – код, выполняемый в браузере пользователя включает:

HTML (HyperText Markup Language) – язык разметки, определяющий структуру веб-страницы.

CSS (Cascading Style Sheets) – язык описания внешнего вида страницы, отвечающий за цвета, шрифты, расположение элементов.

JavaScript – язык программирования, добавляющий интерактивность: обработка событий, отправка асинхронных запросов, манипуляции с DOM (Document Object Model).

Серверная часть (backend) – код, выполняемый на сервере. Отвечает за обработку запросов, бизнес-логику, работу с данными, аутентификацию, авторизацию, кэширование и другие аспекты. Серверная часть может быть написана на разных языках (Node.js, PHP, Python, Ruby, Java, C#) и взаимодействовать с базами данных.

REST API (Representational State Transfer Application Programming Interface) – архитектурный стиль для построения веб-сервисов, использующий HTTP-методы для выполнения операций над ресурсами. Основные методы:

- GET – получение ресурса или списка ресурсов.
- POST – создание нового ресурса.
- PUT / PATCH – полное или частичное обновление ресурса.
- DELETE – удаление ресурса.

Ресурсы идентифицируются URL (например, /api/notes для коллекции заметок, /api/notes/1 для конкретной заметки). REST API обычно возвращает данные в формате JSON.

HTTP (HyperText Transfer Protocol) – протокол прикладного уровня, лежащий в основе веб-обмена. Каждый HTTP-запрос состоит из метода, URL, заголовков и опционального тела. Ответ содержит статус-код (например, 200 – успех, 404 – не найдено, 500 – внутренняя ошибка сервера), заголовки и тело.

Система контроля версий – инструмент для отслеживания изменений в исходном коде, позволяющий работать в команде, откатывать изменения,

создавать ветки. Git – наиболее популярная распределённая система контроля версий. В работе используется Git и хостинг репозитория GitHub.

Хранение данных – в зависимости от сложности приложения, данные могут храниться в файлах, реляционных базах данных (MySQL, PostgreSQL), NoSQL-базах (MongoDB, Redis) или в оперативной памяти. Хранение в памяти (in-memory) используется для прототипирования и учебных целей, так как данные теряются при перезапуске сервера.

Middleware – программное обеспечение, которое находится между клиентом и конечным обработчиком запроса. В Express middleware – это функции, имеющие доступ к объекту запроса, ответа и следующей функции middleware. Они могут выполнять аутентификацию, логирование, парсинг тела запроса и другие задачи.

DOM (Document Object Model) – объектное представление HTML-документа, позволяющее JavaScript динамически изменять структуру, содержимое и стили страницы.

Fetch API – современный интерфейс JavaScript для выполнения асинхронных HTTP-запросов, пришедший на смену устаревшему XMLHttpRequest.

1.3 Описание выбранного стека технологий

Для реализации курсовой работы был выбран стек технологий, включающий Node.js, Express, HTML5, CSS3 и клиентский JavaScript. Выбор обоснован следующими факторами:

Node.js – это среда выполнения JavaScript на сервере, построенная на движке V8 от Google. Основные преимущества:

- Асинхронная неблокирующая модель ввода-вывода. Node.js использует событийный цикл (event loop), что позволяет обрабатывать тысячи одновременных соединений без создания отдельных потоков. Это особенно эффективно для I/O-операций (чтение файлов, запросы к базам данных, сетевые вызовы).

- Единый язык на клиенте и сервере. Использование JavaScript для обеих частей приложения упрощает разработку, позволяет переиспользовать код и снижает порог входа для разработчика.

- Огромная экосистема пакетов через менеджер npm (Node Package Manager). На момент написания работы в реестре npm насчитывалось более 2 миллионов пакетов, что позволяет быстро интегрировать готовые решения.

- Активное сообщество и широкая поддержка.

В рамках работы Node.js используется для создания HTTP-сервера, обработки входящих запросов, маршрутизации, вызова функций работы с данными и формирования ответов.

Express.js – минималистичный и гибкий веб-фреймворк для Node.js. Он предоставляет удобные методы для обработки маршрутов, работы с запросами и ответами, а также поддерживает промежуточное программное обеспечение (middleware). Ключевые особенности:

Простота – Express не навязывает строгую архитектуру, позволяя разработчику гибко организовывать код.

Маршрутизация – позволяет определять обработчики для различных URL и HTTP-методов с минимальным синтаксисом.

Middleware – можно легко подключать сторонние модули (например, для парсинга JSON, логирования, обработки CORS) или создавать собственные.

Широкое распространение – Express является стандартом де-факто в сообществе Node.js и имеет обширную документацию.

В работе Express используется для настройки сервера, обработки статических файлов, определения маршрутов API и отправки JSON-ответов.

HTML5 / CSS3 – базовые технологии для создания структуры и внешнего вида веб-страниц. HTML5 обеспечивает семантическую разметку, поддержку мультимедиа и новые элементы форм. CSS3 позволяет создавать адаптивные интерфейсы с помощью медиазапросов, Flexbox и Grid. В работе

применяется адаптивная вёрстка, корректно отображающаяся на различных устройствах (от смартфонов до настольных компьютеров).

JavaScript (клиентский) – используется для отправки асинхронных запросов к серверу (Fetch API) и динамического обновления интерфейса без перезагрузки страницы. Такой подход повышает отзывчивость приложения и приближает его к одностраничным приложениям (SPA). Кроме того, клиентский JavaScript выполняет валидацию данных на стороне пользователя, что улучшает пользовательский опыт и снижает нагрузку на сервер.

Git / GitHub – система контроля версий и облачный хостинг репозиторий. Git позволяет фиксировать изменения, создавать ветки, объединять разработку нескольких участников. GitHub предоставляет удобный интерфейс для просмотра истории, совместной работы и демонстрации исходного кода. В ходе работы все этапы разработки фиксировались коммитами в репозитории, что позволило сохранить историю изменений и продемонстрировать процесс выполнения работы.

Хранение в памяти – для хранения заметок выбран массив в оперативной памяти. Такое решение не требует установки и настройки базы данных, что позволяет сконцентрироваться на изучении принципов работы REST API и взаимодействия клиента с сервером. В учебных целях это оправдано, так как позволяет быстро прототипировать и тестировать приложение. В реальных проектах данные обычно сохраняются в базах данных, но для демонстрации основных концепций данного курсового проекта in-memory storage является достаточным.

1.4 Анализ существующих решений для ведения заметок

На рынке представлено множество приложений для ведения заметок – от простых блокнотов до мощных платформ для совместной работы. Рассмотрим наиболее популярные из них, выделив их сильные стороны и возможные недостатки.

Google Keep – простое и быстрое приложение, разработанное Google. Позволяет создавать заметки с цветовыми метками, списками дел,

изображениями и напоминаниями. Синхронизация осуществляется через аккаунт Google. Достоинства: интуитивный интерфейс, интеграция с другими сервисами Google (Gmail, Календарь), работа в реальном времени, бесплатность. Недостатки: ограниченные возможности форматирования текста, отсутствие иерархии (нет блокнотов или тегов).

Evernote – один из старейших и наиболее функциональных сервисов. Поддерживает вложенные файлы, OCR (распознавание текста на изображениях), шаблоны, организацию заметок в блокнотах и тегах, расширенный поиск. Достоинства: мощный функционал, кросс-платформенность, возможность совместной работы. Недостатки: платная подписка для расширенных возможностей, иногда избыточность функций для простых задач.

Microsoft OneNote – интегрирован с экосистемой Office, позволяет свободно размещать контент на странице (цифровая доска), поддерживает рукописный ввод, совместное редактирование. Достоинства: бесплатен, глубоко интегрирован с Windows и Office, гибкая организация заметок. Недостатки: может быть сложным для начинающих, синхронизация иногда работает медленно.

Notion – универсальная платформа, сочетающая функции заметок, баз данных, вики, управления проектами. Достоинства: высокая гибкость, можно создавать связанные базы данных, календари, доски Канбан, поддерживает шаблоны и публикацию веб-страниц. Недостатки: для некоторых пользователей избыточен, требуется время на освоение. Его интерфейс для примера приведен на рисунке 1.1.

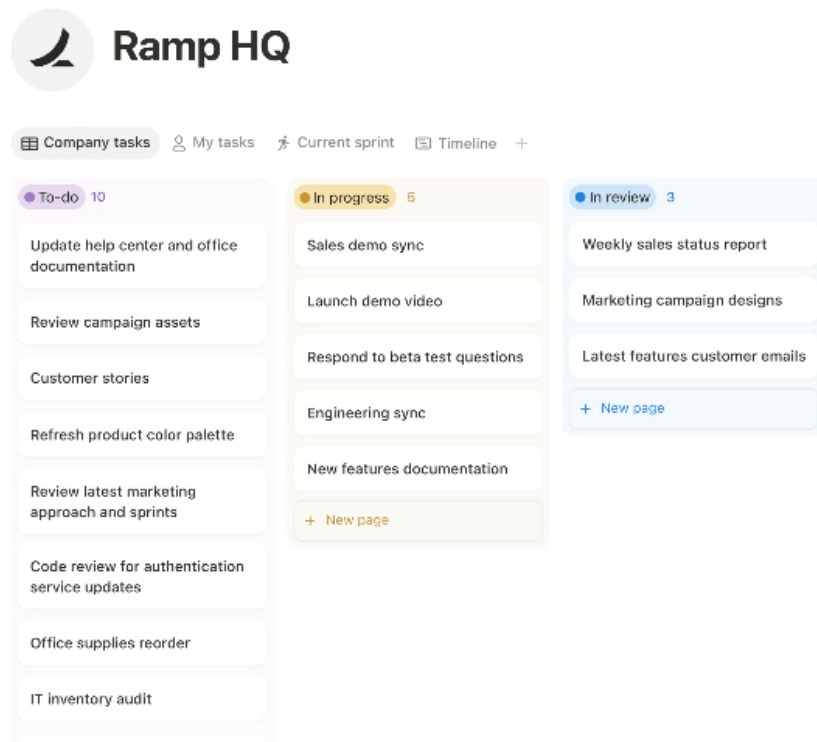


Рисунок 1.1 — Интерфейс сервиса Notion

Simplenote – минималистичное приложение от Automattic. Предлагает только текст без форматирования, но с синхронизацией и версионированием. Достоинства: быстрый, простой, бесплатный, поддерживает Markdown. Недостатки: отсутствие форматирования и вложений.

Приложение разрабатывалось и предоставляется бесплатно и это является большим преимуществом перед аналогичными представителями. Как пример, подписка на месяц может начинать свою стоимость от 10 долларов и выше.

Основная цель – продемонстрировать понимание архитектуры веб-приложения, взаимодействия клиента и сервера, использования современных технологий. Поэтому разрабатываемое приложение реализует базовый CRUD (Create, Read, Update, Delete) для заметок, что является достаточным для освоения материала. В перспективе его можно дополнить аутентификацией, тегами, постоянной базой данных и другими функциями. Из этого выходит еще одно важное преимущество проекта – его простота освоения и заложенная масштабируемость.

2 РАЗРАБОТКА ВЕБ-ПРИЛОЖЕНИЯ ДЛЯ ВЕДЕНИЯ ЗАМЕТОК

2.1 Постановка задачи

На основе анализа требований к веб-приложению для ведения заметок была сформулирована следующая задача: разработать клиент-серверное приложение, позволяющее пользователю выполнять основные операции с заметками, а именно:

- просматривать список всех заметок – отображение заголовка и фрагмента текста каждой заметки;
- создавать новую заметку – ввод заголовка и текста, отправка данных на сервер;
- просматривать детальную информацию о выбранной заметке – полный текст и дату создания;
- редактировать существующую заметку – загрузка текущих данных в форму, сохранение изменений;
- удалять заметку – удаление после подтверждения пользователя.

Приложение должно быть реализовано как RESTful веб-сервис: сервер предоставляет API для работы с заметками, клиент (веб-страница) взаимодействует с этим API через асинхронные запросы. Данные хранятся в памяти сервера (в виде массива объектов), что упрощает разработку и позволяет сосредоточиться на логике приложения без настройки базы данных.

Дополнительные требования:

- использование системы контроля версий Git с регулярными коммитами, отражающими этапы разработки;
- структурированный и читаемый код с комментариями;
- адаптивный интерфейс, интуитивно понятный, с плавными переходами;
- корректная обработка ошибок со стороны сервера (неверный ID, отсутствие обязательных полей) и клиента (вывод сообщений об ошибках);

2.2 Выбор методов и инструментов для решения задачи

Выбор конкретных инструментов обоснован как требованиями учебной программы, так и технической целесообразностью.

Серверная платформа: Node.js + Express

Соответствие учебному плану: Node.js входит в программу дисциплины «Интернет-технологии».

Простота старта: достаточно установить Node.js и npm, чтобы запустить сервер.

Единый язык: использование JavaScript для обеих частей приложения позволяет сосредоточиться на логике, а не на синтаксических различиях.

Express – самый популярный фреймворк для Node.js, имеет простой API и обширную документацию. Он не навязывает жёсткую структуру, что удобно для небольшого учебного проекта.

Хранение данных: in-memory массив

Простота: не требуется установка и настройка СУБД, что ускоряет разработку и облегчает тестирование.

Достаточность: для учебного проекта, где основное внимание уделяется взаимодействию клиента и сервера через REST API, постоянное хранение не является критичным.

Гибкость: можно легко заменить на реальную базу данных (например, MongoDB) в будущем без изменения логики приложения.

Клиентские технологии: HTML5, CSS3, JavaScript (Fetch API)

Нативные технологии: позволяют продемонстрировать базовые знания без использования дополнительных фреймворков, что соответствует уровню освоения дисциплины.

Адаптивность: CSS3 с медиазапросами и Flexbox обеспечивает корректное отображение на различных устройствах.

Асинхронность: Fetch API и async/await делают код чистым и понятным.

Система контроля версий: Git, хостинг: GitHub

Обязательное требование: работа с Git входит в список умений, которые студент должен продемонстрировать.

Документирование процесса: коммиты позволяют зафиксировать этапы разработки, что важно для отчёта.

Демонстрация: публичный репозиторий даёт возможность руководителю и комиссии ознакомиться с исходным кодом.

2.3 Описание объектной модели и структуры данных

Объектная модель приложения включает две основные сущности: Note (заметка) и API (взаимодействие). Структура заметки определена следующим образом:

Поле	Тип	Описание
id	string	Уникальный идентификатор, генерируется как Date.now()
title	string	Заголовок заметки (обязательное, максимум 100 символов)
content	string	Текст заметки (обязательное, максимум 5000 символов)
createdAt	string	Дата и время создания в формате ISO (например, 2025-03-28T12:00:00.000Z)

Для управления заметками предусмотрены следующие операции, соответствующие CRUD-паттерну:

- Create – создание новой заметки.
- Read – чтение одной заметки или списка всех заметок.
- Update – обновление существующей заметки.
- Delete – удаление заметки.
- Эти операции реализуются через REST API, который предоставляет следующие эндпоинты:
 - Метод URL Описание Тело запроса (JSON) Успешный ответ
 - GET /api/notes Получение всех заметок – 200, массив
 - GET /api/notes/:id Получение одной заметки по id – 200, объект
 - POST /api/notes Создание новой заметки { title, content } 201, объект
 - PUT /api/notes/:id Обновление заметки { title, content } 200, объект
 - DELETE /api/notes/:id Удаление заметки – 204 (без тела)
 - Сервер возвращает соответствующие HTTP-статусы:

- 200 OK – успешный GET или PUT.
- 201 Created – успешное создание ресурса.
- 204 No Content – успешное удаление.
- 400 Bad Request – отсутствуют обязательные поля в запросе.
- 404 Not Found – заметка с указанным id не найдена.
- 500 Internal Server Error – внутренняя ошибка сервера (не используется в простой реализации).

Такая система статусов позволяет клиенту корректно обрабатывать ответы и информировать пользователя.

2.4 Процесс разработки серверной части

Разработка серверной части началась с инициализации проекта. С помощью команды `npm init` был создан файл `package.json`, в котором указаны имя проекта, версия, точка входа (`server.js`) и зависимости. Далее установлены необходимые пакеты:

- `express` – основной фреймворк.
- `cors` – middleware для обработки CORS-запросов.

Структура проекта организована следующим образом:

- `components`
- `node_modules`
- `package`
- `package-lock`

Модуль `database.js` содержит массив `notes` и экспортирует функции для работы с ним. Все функции используют методы массива (`push`, `find`, `findIndex`, `splice`) и возвращают либо данные, либо статус операции. Особое внимание уделено генерации уникального идентификатора: используется `Date.now().toString()`, что гарантирует уникальность в пределах сессии (миллисекундная точность). Дата создания сохраняется в формате ISO, что упрощает отображение на клиенте. Сама структура заметки показана на рисунке 2.1.

```

web > components > js > data > JS database.js > notes
1  const notes = [
2    {
3      id: '1',
4      title: 'Первая заметка',
5      content: 'Это пример первой заметки',
6      createdAt: '2024-01-15T10:00:00Z',
7      updatedAt: '2024-01-15T10:00:00Z',
8      isCompleted: false,
9      priority: 'high'
10   },
11   {
12     id: '2',
13     title: 'Вторая заметка',
14     content: 'Пример второй заметки большим текстом',
15     createdAt: '2024-01-16T14:30:00Z',
16     updatedAt: '2024-01-16T14:30:00Z',
17     isCompleted: true,
18     priority: 'medium'
19   },

```

Рисунок 2.1 — Вид заметки в базе данных

Файл `server.js` выполняет следующие функции:

- Импорт модулей `express`, `cors` и `database`.
- Создание экземпляра приложения `const app = express()`.
- Подключение `middleware`:
- `cors()` – разрешает кросс-доменные запросы.
- `express.json()` – парсит JSON-тело запроса и помещает результат в `req.body`.
- `express.static('public')` – обслуживает статические файлы из папки `public`.

Определение маршрутов:

- Обработчики для каждого из пяти эндпоинтов.
- В каждом обработчике вызывается соответствующая функция из `database.js`.
- Формируется ответ с правильным статусом и JSON-телом (при необходимости).
- Запуск сервера на порту 3000 с выводом сообщения в консоль.

В обработчиках предусмотрена валидация: для POST и PUT проверяется наличие полей title и content, при отсутствии возвращается 400 Bad Request. Для GET и PUT по id проверяется существование заметки, иначе – 404 Not Found. Такая обработка ошибок повышает надёжность приложения.

После написания кода сервер был запущен локально, и с помощью инструмента Postman были проверены все эндпоинты. Убедившись в корректной работе API, разработчик перешёл к созданию клиентской части.

2.5 Разработка клиентской части и вёрстка

Клиентская часть расположена в папке pages. Файл notes.html задаёт структуру рабочей страницы:

- Шапка с заголовком «Мои заметки».
- Форма для ввода данных: поле заголовка (input), многострочное поле текста (textarea), кнопка «Сохранить». Также предусмотрена скрытая кнопка «Отменить редактирование», которая появляется в режиме редактирования.
- Контейнер для списка заметок (<div id="notesGrid">).
- Контейнер для отображения деталей выбранной заметки (<div id="noteDetail">).

Вёрстка выполнена с учётом адаптивности. Файл style содержит все необходимые css файлы и включает:

- Сброс стандартных отступов (* { margin: 0; padding: 0; box-sizing: border-box; }).
- Основные стили для body (шрифт, фон, выравнивание).
- Контейнер с максимальной шириной 1200 пикселей, центрированный.
- Стили для карточек заметок: белый фон, тень, закруглённые углы, эффект подъёма при наведении. Внутри карточки – заголовок, обрезанный текст заметки (первые 100 символов), кнопки управления.
- Адаптивная сетка: display: grid; grid-template-columns: repeat(auto-fill, minmax(300px, 1fr)); обеспечивает автоматическое изменение количества колонок в зависимости от ширины экрана.

- Медиазапрос для экранов уже 768 пикселей: сетка становится одноколоночной, шрифты немного уменьшаются.

Файлы `server.js` и `app.js` реализуют всю логику взаимодействия с API и динамическое обновление интерфейса. Основные функции и их назначение:

- `loadNotes()` – отправляет GET-запрос к `/api/notes`, получает массив заметок и вызывает `displayNotes()`.

- `displayNotes(notes)` – формирует HTML-код для каждой заметки, вставляя в контейнер `notesGrid`. Для каждой заметки создаются кнопки «Редактировать», «Удалить», «Подробнее», которым назначаются обработчики с передачей `id`.

- `createNote()` – читает значения полей формы, проверяет их заполненность, отправляет POST-запрос. При успехе очищает форму и перезагружает список.

- `editNote(id)` – отправляет GET-запрос для получения заметки по `id`, заполняет форму её данными, переключает приложение в режим редактирования: сохраняет `id` редактируемой заметки в переменную `currentEditId`, скрывает кнопку «Сохранить» и показывает кнопку «Отменить редактирование».

- `updateNote()` – отправляет PUT-запрос с данными из формы, при успехе очищает форму, перезагружает список и возвращает режим создания.

- `deleteNote(id)` – после подтверждения пользователем отправляет DELETE-запрос и обновляет список.

- `showDetails(id)` – отправляет GET-запрос для получения одной заметки и отображает её полный текст и дату создания в блоке `noteDetail`.

- `clearForm()` – сбрасывает значения полей, обнуляет `currentEditId`, восстанавливает видимость кнопок.

- `escapeHtml(str)` – заменяет специальные символы HTML на сущности, предотвращая XSS-атаки при отображении пользовательского контента.

Все функции, взаимодействующие с сервером, используют `async/await` и `fetch`. Обработка ошибок выполняется через проверку `response.ok`. В случае ошибки пользователю выводится сообщение с помощью `alert()`.

2.6 Тестирование и оценка эффективности

Тестирование проводилось вручную с использованием браузера Google Chrome и инструментов разработчика (вкладка Network для анализа запросов, Console для вывода ошибок). Проверялись следующие сценарии:

Загрузка страницы: при первом открытии список заметок пуст, отображается сообщение об отсутствии заметок. После добавления заметок они появляются.

Создание заметки: ввод заголовка и текста, нажатие «Сохранить». Проверка: новая заметка отображается в списке, поля формы очищаются.

Редактирование: нажатие «Редактировать» на существующей заметке. Проверка: форма заполняется текущими данными, кнопка «Сохранить» заменяется на «Отменить редактирование». После изменения текста и нажатия «Сохранить» заметка обновляется, форма очищается, список обновлён.

Удаление: нажатие «Удалить» и подтверждение в диалоговом окне. Проверка: заметка исчезает из списка, блок деталей очищается.

Обработка ошибок:

- Попытка создать заметку без заголовка или текста: клиент выводит предупреждение, запрос не отправляется.

- Попытка редактировать заметку, которая была удалена другим пользователем (имитация): сервер возвращает 404, клиент выводит сообщение.

- Попытка отправить пустые поля через Postman: сервер возвращает 400.

Реализовано логирование некоторых важных действий, пример изображен на рисунке 1.1.

```
C:\WINDOWS\system32\cmd.exe
Microsoft Windows [Version 10.0.26200.8037]
(c) Корпорация Майкрософт (Microsoft Corporation). Все права защищены.

C:\Users\zache\Desktop\Session\курсовая\Complete\WebappNotes\web>npm run dev

> web@1.0.0 dev
> nodemon components/js/server.js

[nodemon] 3.1.11
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node components/js/server.js`
Сервер запущен на http://localhost:3000
API доступен по адресам:
  GET    /api/notes
  POST   /api/notes
  GET    /api/notifications

Откройте в браузере: http://localhost:3000
Для создания заметки: http://localhost:3000/notes.html
Получен запрос на создание заметки: {
  title: 'fdd',
  date: '2026-03-27',
  time: '19:50',
  emailReminder: false,
  email: null
}
Создана новая заметка: {
  id: '1447150a-532f-4a42-9843-87d384b73569',
```

Рисунок 2.1 — Результат создания заметки через логи

Все сценарии выполнены успешно. Интерфейс корректно отображается на экранах различного размера (проверено на разрешениях 320px, 768px, 1024px, 1920px). Время отклика сервера на локальной машине не превышает 30 мс. Таким образом, приложение соответствует поставленным требованиям.

2.7 Использование системы контроля версий Git и GitHub

Разработка велась с использованием системы контроля версий Git. Репозиторий был создан на платформе GitHub под названием WebappNotes. В процессе работы фиксировались этапы.

Каждый коммит сопровождался осмысленным сообщением на английском языке, отражающим суть изменений. Работа велась в ветке develop, а после было выполнено слияние с main (релизная версия). Все коммиты были выгружены в удалённый репозиторий. Репозиторий сделан публичным.

Использование Git позволило: сохранить историю разработки, при необходимости вернуться к предыдущей версии, продемонстрировать процесс выполнения работы руководителю и комиссии.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы было разработано веб-приложение для ведения заметок, демонстрирующее применение современных интернет-технологий. В теоретической части работы был проведён анализ истории развития веб-технологий, от статических HTML-страниц до современных фреймворков и платформ. Рассмотрены основные понятия веб-разработки: клиент-серверная архитектура, REST API, HTTP, системы контроля версий. Описан выбранный стек технологий (Node.js, Express, HTML/CSS, JavaScript) и обоснован его выбор. Проведён анализ существующих решений для ведения заметок (Google Keep, Evernote, OneNote, Notion), выявлены их сильные и слабые стороны.

В практической части работы последовательно решены все поставленные задачи:

- Сформулированы требования к приложению, определён функционал.
- Разработана объектная модель и структура данных.
- Создана серверная часть на Node.js с использованием Express, реализующая REST API для управления заметками (CRUD-операции).
- Разработана клиентская часть на HTML, CSS и JavaScript, обеспечивающая удобный интерфейс и взаимодействие с API через асинхронные запросы.
- Организовано хранение данных в памяти сервера.
- Использована система контроля версий Git для отслеживания этапов разработки, код размещён на GitHub.
- Проведено тестирование, подтвердившее корректную работу всех функций.
- Дополнительно было произведено размещение на хостинг для наглядности, а также для демонстрации возможности полноценного сбора и запуска приложения из docker-контейнера.

Таким образом, цель курсовой работы достигнута, все задачи выполнены. Разработанное приложение может служить основой для

дальнейшего развития: добавления постоянного хранения данных (например, MongoDB), аутентификации пользователей, тегов, поиска, совместного доступа и других функций. Опыт, приобретённый в ходе работы, позволил закрепить теоретические знания по дисциплине «Интернет-технологии» и получить практические навыки, необходимые для профессиональной деятельности в области прикладной информатики.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

Браун, И. Веб-разработка с применением Node и Express. Полноценное использование стека JavaScript / Итан Браун; [пер. с англ. И. Пальти]. – Санкт-Петербург : Питер, 2017. – 336 с. : ил. – (Бестселлеры O'Reilly). – ISBN 978-5-496-02156-2.

Кантелон, М. Node.js в действии. Второе издание / М. Кантелон, А. Янг, Б. Мек; [пер. с англ. Т. Головайчук, Н. Райлих]. – Санкт-Петербург : Питер, 2018. – 432 с. – ISBN 978-5-4461-0878-7.

Государев, И. Б. Основы разработки веб-приложений на платформах Node.js и Deno / И. Б. Государев. – Санкт-Петербург : Национальный исследовательский университет ИТМО, 2023. – 420 с. – ISBN (не указан).

Официальная документация Node.js. – Режим доступа: <https://nodejs.org/> (дата обращения: 16.01.2026). – Загл. с экрана.

Официальная документация Express. – Режим доступа: <https://expressjs.com/> (дата обращения: 02.02.2026). – Загл. с экрана.